Agent Concepts and MCP Basics

Explainer + evidence of a working Gmail MCP connector

Agents vs. Workflows:

An **agent** is a system that works independently based on a goal by dynamically choosing its own tools and execution steps. Instead of following a script, an agent looks at what it's trying to accomplish, decides what to do next, checks the result, and decides again, looping and adjusting until the goal is met or it hits a limit. Basically, it changes its steps as it goes, instead of following one set path.

A **workflow** is a system of pre-defined, fixed steps through which a system runs. Every input follows the same sequence, in the same order, with no branching based on what happens mid-run. A human (or a rule) decides the path ahead of time, and the system just executes it. Workflows are more predictable and easier to debug, but they can't adapt to something the designer didn't anticipate.

The distinction matters because "agent" gets used as marketing shorthand for almost anything AI-powered these days, even a chatbot with a system prompt. The real test isn't "does it use AI"; it's "does the system decide its own next step, or did a human decide it in advance?"

Classifying My FL-04 Pipeline

My FL-04 workflow: draft, critique, revise for job application proposals is a **workflow**, not an agent. Every run follows the same fixed sequence: gather requirements, draft, critique, and revise. Nothing about the path changes based on what the model finds; I'm the one deciding when to move to the next step and what to do with the critique's feedback.

More specifically, it's a combination of two patterns. Steps 1 to 2 (gather → draft) are prompt chaining: a one-way handoff where one step's output becomes the next step's input, no evaluation or looping involved. Steps 2 through 4 (draft → critique → revise) form an evaluator-optimiser pattern: one step generates a draft, a separate step evaluates it against explicit criteria (the requirements table from Step 1), and a third step optimises the draft based on that evaluation. The difference from a true evaluator-optimiser agent is that the loop isn't autonomous - the model doesn't decide on its own whether to send the draft back for another round of critique. I do. I read the critique memo, decide what's worth acting on, and manually trigger Step 4. The model never gets to say "this still isn't good enough, let's go again" on its own.

What MCP Is

MCP is an open-source protocol for connecting AI applications with external systems. It solves a real gap: a language model on its own only knows what's in its training data and whatever text gets pasted into the conversation, it can't check your actual inbox, browse a live repository, or read a file on your computer. MCP gives it a standard way to reach out and interact with those external systems instead of guessing.

MCP defines three primitives. Tools are actions the model can actively call - functions like "search my inbox" or "list my repositories" that the model decides to invoke based on what the user asked. Resources are data the application can expose to the model as context, more like read-only reference material than something the model actively triggers. Prompts are pre-written templates a server can offer to structure how a task gets approached, so the interaction isn't starting from a blank slate every time.

In practice, when I ran my three Gmail tasks, I was watching the tools primitive in action - each prompt triggered a visible tool call where Claude reached into my actual inbox rather than answering from memory.

I actually ran into a good example of why this matters for evaluating real products: I initially tried to connect a GitHub connector that showed as "connected" in settings, but it failed to actually authenticate when Claude tried to use it. Switching to Gmail worked cleanly - all three tasks (searching by sender, cross-referencing job-application-related emails, and counting/filtering unread messages) triggered visible tool calls and returned accurate data. The lesson: a connector showing as "connected" in the UI doesn't guarantee it's actually functional, which is worth checking before trusting any agent-product demo. A second, smaller example of the same caution shows up in Task 3 below, where Claude reported "0 spam emails" - a claim worth a manual gut-check rather than automatic trust, since spam folders aren't always included in what's queryable.

What FL-04 Would Need to Become an Agent

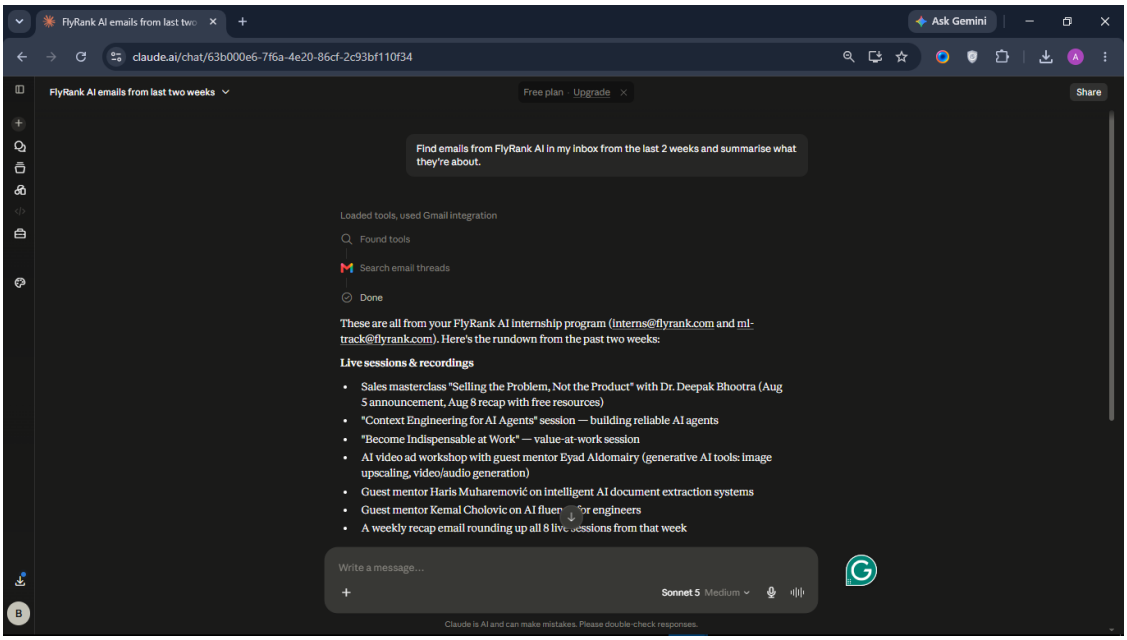
The core gap is the loop-control decision in Step 4. Right now, I read the critique and decide whether the concerns matter enough to warrant another draft. A true agentic version would hand that judgment to the model itself: after critique, the model would decide on its own whether the draft is good enough to stop, or whether it needs another revision cycle, and it would keep looping - critiquing its own revision, deciding again - until it judged the draft ready, without me manually re-triggering each step. That single change, an autonomous stopping decision instead of a human one, is what would turn this from a fixed workflow into a genuine evaluator-optimiser agent.

Evidence: Gmail MCP Connector — Three Live Tasks

Connector used: Gmail (connected via Claude Settings → Connectors). Note: a GitHub connector was attempted first and showed as "connected" in settings, but failed to authenticate when actually invoked - switched to Gmail, which worked reliably for all three tasks below. Each screenshot shows the visible tool-call trail (e.g. "Used Gmail integration" → "Search email threads" → "Done") preceding Claude's answer, confirming live data retrieval rather than a memory-based response.

Task 1 — Search by sender

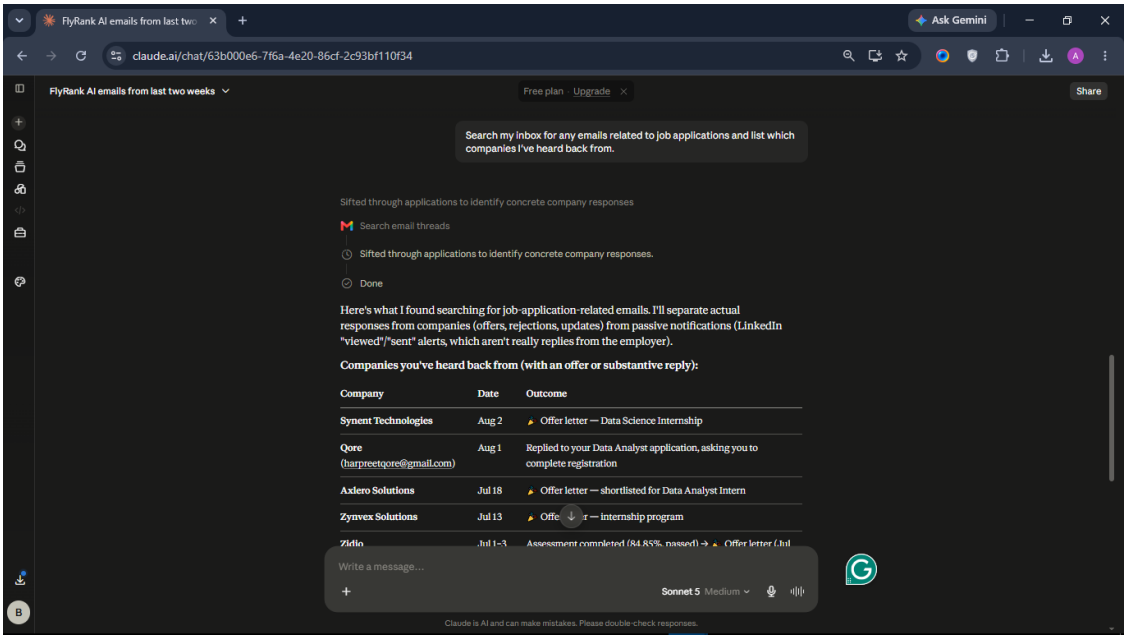
Prompt: "Find emails from FlyRank AI in my inbox from the last 2 weeks and summarise what they're about."



Tool-call trail visible: Loaded tools → used Gmail integration → Search email threads → Done, followed by a real summary of FlyRank AI internship emails from the actual inbox.

Task 2 — Cross-reference job applications

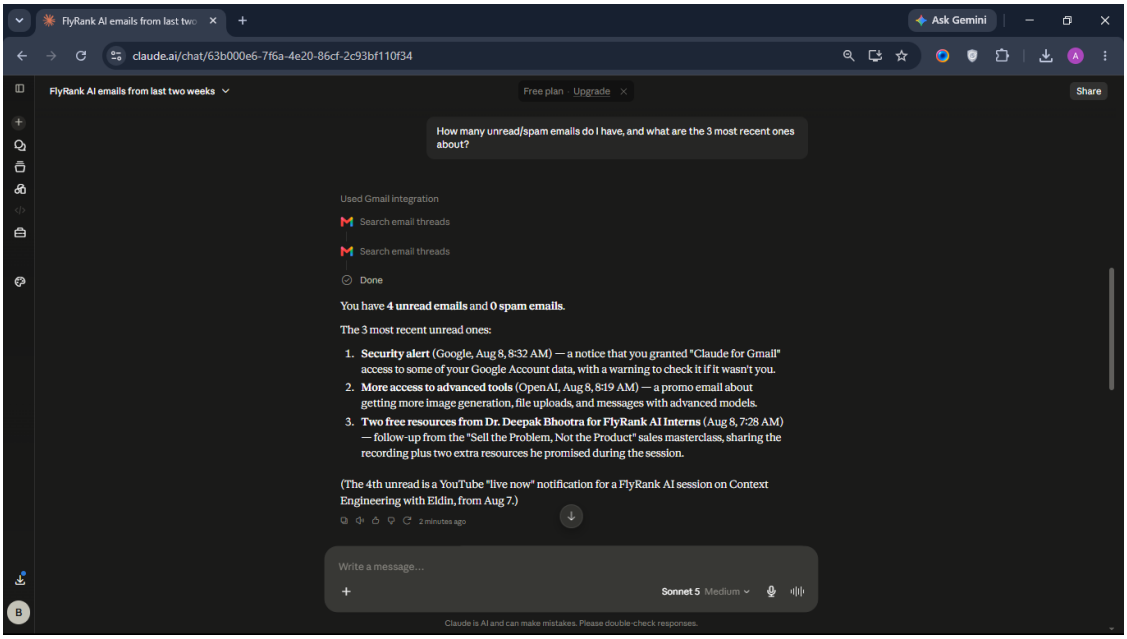
Prompt: "Search my inbox for any emails related to job applications and list which companies I've heard back from."



Tool-call trail visible: Search email threads → Done, followed by a real table of companies, dates, and outcomes pulled from actual email threads — information chat alone could not have known.

Task 3 — Count and filter unread messages

Prompt: "How many unread/spam emails do I have, and what are the 3 most recent ones about?"



Tool-call trail visible: Used Gmail integration → Search email threads (x2) → Done, followed by an accurate unread count and summary of the 3 most recent messages.